SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – APPLICATION BASED QUESTIONS

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO2	**Assessment:**	Application Based Questions, In-class
Weightage:	_____ 40% of CIA	**Duration / Marks:**	60 mins
CO2 Statement:	Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)		
Student Name:	S Sandhiya	**Reg. No.:**	192321015
Section / Batch:	Slot D	**Date:**	22-10-2026

Code of Conduct

I S Sandhiya(192321015) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S Sandhiya

Instructions

- Read each question carefully before answering (in-Class Assessment)
- Each question is a real-world application scenario. Identify the AI technique that fits, apply it step by step, and justify every decision.

- Questions are drawn from Informed Search (Greedy, A*, heuristics), Local Search and Online Search, Constraint Satisfaction Problems, and Adversarial Search (Minimax, Alpha-Beta, evaluation functions).
- Show intermediate tables (open lists, domains, α/β values) and state the final answer clearly.
- Marks will be awarded using the Application Based Questions rubric: understanding of problem context, application of concepts, problem-solving approach, accuracy of solution and clarity.
- Upload your work in GCR and Github.

Section / Topic	Focus	Question Nos.
1. Informed Search Strategies	Greedy best-first search, A* search, heuristic functions (admissibility, consistency)	**Q1**
2. Local Search, Online Search & Constraint Satisfaction	Local search and optimization problems, online search agent, CSP (MRV, degree heuristic, forward checking)	**Q2, Q3**
3. Game Playing: Adversarial Search	Games, minimax algorithm, alpha-beta pruning, evaluation functions	**Q4**

Rubrics for Application Based Questions

Criteria	Weightage	Level 4 – Exemplary (100% of marks)	Level 3 – Proficient (75%)	Level 2 – Developing (50%)	Level 1 – Beginning (25%)
Understanding of Problem Context	**20%**	Clearly understands the problem and accurately identifies	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem

		all requirements			
Application of Concepts	**30%**	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	**20%**	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	**20%**	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	**10%**	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	**Understanding of Problem Context (2)**	**Application of Concepts (3)**	**Problem-Solving Approach (2)**	**Accuracy of Solution (2)**	**Clarity (1)**	**Sub. Total (10)**	**Total %**
1							
2							
3							

4							

Faculty In-charge	**Course Coordinator**	**Program Director**
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Annexure A – Application Based Questions

Question 1: Campus Delivery Robot – Greedy Best-First vs A*

Topic: Informed search strategies, heuristic functions | 15 minutes

A campus delivery robot must carry a parcel from the Store (S) to the Hostel (G). The one-way road network and the robot's heuristic estimate h(n) of the remaining travel cost to G (in minutes) are given below.

From	**To**	**Cost**
S	A	1
S	B	4
A	C	2
A	D	5
B	G	5
C	D	1
D	G	2

Node	S	A	B	C	D	G
h(n)	6	5	3	3	2	0

Assume ties are broken alphabetically and that a node is re-opened if a cheaper path to it is found.

(a) Apply Greedy Best-First Search from S to G. Write the order in which nodes are expanded, the path returned and its total cost.

(b) Apply A* search. Show the open list with g(n), h(n) and f(n) after each expansion, the order of expansion, and the final path and cost.

(c) Check whether h(n) is admissible and consistent by verifying the required condition for every node and every road.

(d) Compare the outputs of (a) and (b). Explain why Greedy Best-First Search returned a different result and which algorithm you would deploy for the robot, with justification.

Question 1: Campus Delivery Robot – Greedy Best-First Search vs A*

Given data

Road network

From	To	Cost
S	A	1
S	B	4
A	C	2
A	D	5
B	G	5
C	D	1
D	G	2

Heuristic values

Node	\(h(n)\)
S	6
A	5

B	3
C	3
D	2
G	0

The algorithms use the following evaluation functions:

- Greedy Best-First Search: $f(n)=h(n)$
- A Search:* $f(n)=g(n)+h(n)$

Here, $g(n)$ is the actual cost from the start node, and $h(n)$ is the estimated remaining cost to the goal.

(a) Apply Greedy Best-First Search

Greedy Best-First Search selects the node with the smallest heuristic value $h(n)$, without considering the actual path cost already incurred.

Step-by-step process

1. Start at S. The successors are A $(h=5)$ and B $(h=3)$. Select B.
2. Expand B. Its successor is G $(h=0)$. Select G.
3. G is the goal, so the search terminates.

Order of expansion: S → B → G

Path returned: S → B → G

(b) Apply A* Search

A* selects the node with the smallest $f(n)=g(n)+h(n)$. If a cheaper path to a node is found, its cost and parent are updated.

Step-by-step calculation

Step 1: Expand S

- A: $g=1,\ h=5,\ f=1+5=6$
- B: $g=4,\ h=3,\ f=4+3=7$

Open list:

Node	**$g(n)$**	**$h(n)$**	**$f(n)$**
A	1	5	6

B	4	3	7

Select A.

Step 2: Expand A

- C: $g=1+2=3,\ h=3,\ f=6$
- D: $g=1+5=6,\ h=2,\ f=8$

Open list:

Node	**$g(n)$**	**$h(n)$**	**$f(n)$**
C	3	3	6
B	4	3	7
D	6	2	8

Select C.

Step 3: Expand C

- D through C: $g=3+1=4,\ h=2,\ f=6$
- The new path to D costs 4, which is cheaper than the previous cost of 6. Therefore, update D.

Open list:

Node	**$g(n)$**	**$h(n)$**	**$f(n)$**
D	4	2	6
B	4	3	7

Select D.

Step 4: Expand D

- G: $g=4+2=6,\ h=0,\ f=6$

Open list:

Node	**$g(n)$**	**$h(n)$**	**$f(n)$**
G	6	0	6
B	4	3	7

Select G. Since G is the goal, the search terminates.

Final results

- Order of expansion: S → A → C → D → G
- Optimal path: S → A → C → D → G
- Total cost: $1+2+1+2=\boxed{6\text{ minutes}}$

(c) Check whether the heuristic is admissible and consistent

1. Admissibility

A heuristic is admissible if it never overestimates the actual minimum cost to reach the goal:

$$ h(n)\leq h^*(n) $$

where $h^*(n)$ is the true shortest distance from node n to G.

Node	$h(n)$	Actual shortest cost $h^*(n)$	Admissible?
S	6	6	Yes
A	5	5	Yes
B	3	5	Yes
C	3	3	Yes
D	2	2	Yes
G	0	0	Yes

All heuristic values are less than or equal to the actual shortest costs. Therefore, the heuristic is admissible.

2. Consistency

A heuristic is consistent if, for every road from node u to node v,

$$ h(u)\leq c(u,v)+h(v) $$

where $c(u,v)$ is the road cost.

Road	Verification	Result
S → A	$6\leq1+5=6$	True
S → B	$6\leq4+3=7$	True
A → C	$5\leq2+3=5$	True
A → D	$5\leq5+2=7$	True
B → G	$3\leq5+0=5$	True

C → D	\(3\leq1+2=3\)	True
D → G	\(2\leq2+0=2\)	True

Every road satisfies the consistency condition. Therefore, the heuristic is consistent.

(d) Compare Greedy Best-First Search and A*

Feature	**Greedy Best-First Search**	**A* Search**
Evaluation function	\(f(n)=h(n)\)	\(f(n)=g(n)+h(n)\)
Considers actual path cost	No	Yes
Nodes expanded	S, B, G	S, A, C, D, G
Path returned	S → B → G	S → A → C → D → G
Total cost	9 minutes	6 minutes
Optimal solution guaranteed?	No	Yes, with an admissible heuristic under standard conditions

Question 2: Wi-Fi Access Point Placement – Local Search and Online Search

Topic: Local search algorithms and optimization problems, online search agent | 15 minutes

A technician wants to place a Wi-Fi access point at the best position along a corridor with 10 candidate positions x = 0 to 9. The measured signal quality f(x) (higher is better) is:

x	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
f(x)	2	5	9	6	4	7	12	10	8	3

A move shifts the access point to an adjacent position (x − 1 or x + 1). Steepest-ascent hill climbing moves to the better neighbour and stops when no neighbour is strictly better.

(a) Run steepest-ascent hill climbing from the start positions x = 0, x = 4 and x = 9. Show each step and the final position and signal quality.

(b) Identify all local maxima and the global maximum. If the start position is chosen uniformly at random from the 10 positions, find the probability that one run reaches the global maximum, the probability that at least one of 3 random-restart runs does, and the expected number of runs needed.

(c) A simulated annealing search is at x = 2 and proposes the move to x = 3. Compute the acceptance probability e^(Δ/T) at T = 2 and at T = 1, and explain what this does to the chance of escaping the local maximum as T falls.

(d) The same search is later done by a robot that cannot see the whole corridor, can only measure f(x) where it stands, and cannot undo exploration for free. Explain why this is an online search problem, and state what the agent must remember to avoid cycling.

(a) Steepest-ascent hill climbing

The signal quality values are:

Position x	Signal quality $f(x)$
0	2
1	5
2	9
3	6
4	4
5	7
6	12
7	10
8	8
9	3

Steepest-ascent hill climbing moves to an adjacent position only if its signal quality is strictly higher than the current position.

Start 1: $x=0$

Step	Position		Signal quality	Decision
1	0	2		Move to 1
2	1	5		Move to 2
3	2	9		Stop; both neighbours are worse

- Final position: $x=2$

- Final signal quality: $f(2)=9$

Start 2: $x=4$

Step	Position	Signal quality	Decision
1	4	4	Move to 5
2	5	7	Move to 6
3	6	12	Stop; both neighbours are worse

- Final position: $x=6$

- Final signal quality: $f(6)=12$

Start 3: $x=9$

Step	Position	Signal quality	Decision
1	9	3	Move to 8
2	8	8	Move to 7
3	7	10	Move to 6
4	6	12	Stop; both neighbours are worse

- Final position: $x=6$

- Final signal quality: $f(6)=12$

(b) Local maxima, global maximum and probabilities

A local maximum is a position whose signal quality is strictly higher than that of its available neighbours. The global maximum is the position with the highest signal quality overall.

The local maxima are:

- $x=2,\ f(2)=9$

- $x=6,\ f(6)=12$

The global maximum is $x=6$, with signal quality 12.

1. Probability that one run reaches the global maximum

Hill climbing reaches the global maximum when the starting position is $x=3,4,5,6,7,8,$ or 9.

There are 7 successful starting positions out of 10.

$$
P(\text{success})=\frac{7}{10}=\boxed{0.7}
$$

Therefore, the probability is 70%.

2. Probability that at least one of 3 random-restart runs succeeds

Assuming the three starting positions are chosen independently and uniformly at random:

$$
\begin{aligned}
P(\text{at least one success}) \\
&=1-P(\text{all fail}) \\
&=1-\left(1-\frac{7}{10}\right)^3 \\
&=1-(0.3)^3 \\
&=\boxed{0.973}
\end{aligned}
$$

Therefore, the probability is 97.3%.

3. Expected number of runs needed

Each independent run has success probability $p=0.7$. The number of runs until the first success follows a geometric distribution.

$$
E[N]=\frac{1}{p}=\frac{1}{0.7}
=\boxed{1.43\text{ runs}}
$$

Therefore, the expected number of runs needed is approximately 1.43 runs.

(c) Simulated annealing

The robot is at $x=2$, where $f(2)=9$, and proposes moving to $x=3$, where $f(3)=6$.

The change in signal quality is:

$$
\Delta=f(3)-f(2)=6-9=-3
$$

Since this is a worse move, simulated annealing accepts it with probability:

$$
P=e^{\Delta/T}=e^{-3/T}
$$

At temperature $T=2$:

$$
P=e^{-3/2}=e^{-1.5}
\approx\boxed{0.2231}
$$

Acceptance probability = 22.31%.

At temperature $T=1$:

$$
P=e^{-3/1}=e^{-3}
\approx\boxed{0.0498}
$$

Acceptance probability = 4.98%.

Explanation: At higher temperatures, simulated annealing is more willing to accept worse moves, helping it escape local maxima. As the temperature falls, the probability of accepting worse moves decreases. Thus, the search gradually becomes less exploratory and more focused on improving the solution.

(d) Online search problem

This becomes an online search problem because the robot cannot see the entire corridor or know all signal values in advance. It can measure the signal quality only at its current position and must explore the corridor while making decisions.

Unlike offline search, the robot must learn about the environment through its actions, and exploration may have a real cost.

The agent should remember:

1. Current position: Where the robot is standing.

2. Visited positions: Which locations it has already explored.

3. Measured signal values: The observed signal quality at each visited position.

4. Previous actions or movement history: To avoid repeatedly moving between the same positions.

5. Known neighbouring positions: Which adjacent locations have already been explored and what their measured values are.

Keeping this information helps the robot avoid cycling, reduce unnecessary movement, and make better decisions with incomplete information.

Question 3: Exam Timetable Scheduling – Constraint Satisfaction Problem

Topic: Constraint satisfaction problem (MRV, degree heuristic, forward checking) | 15 minutes

Five exams A, B, C, D and E must each be given one slot from {1, 2, 3}. Two exams that share students must not be in the same slot. The pairs sharing students are: A–B, A–C, A–D, B–C, B–D, C–E, D–E. Because of invigilator availability, exam A can only be in slots 1 or 2, and exam E only in slots 1 or 2.

Exam	Initial domain
A	{1, 2}
B	{1, 2, 3}
C	{1, 2, 3}
D	{1, 2, 3}
E	{1, 2}

Rules for the search: choose the variable using Minimum Remaining Values (MRV), break ties with the degree heuristic, then alphabetically; try values in ascending order; use forward checking after every assignment.

(a) Draw the constraint graph (list its edges) and compute the degree of every exam.

(b) Select the first exam to assign using MRV and the degree heuristic, justify the choice, assign its first value, and show the domains of all exams after forward checking.

(c) Continue the search with the same rules, showing the chosen exam, its value and the updated domains at each step, until a complete assignment is found. Verify it against every constraint.

(d) Interpret: explain how MRV, the degree heuristic and forward checking reduced the search effort here, and what forward checking would signal if a domain became empty.

(a) Constraint graph and degree of every exam

In a constraint graph, each exam is represented by a vertex, and an edge connects two exams that share students and therefore cannot be scheduled in the same slot.

Edges of the constraint graph:

$$ E=\{(A,B),(A,C),(A,D),(B,C),(B,D),(C,E),(D,E)\} $$

Degree of each exam: The degree is the number of edges connected to a vertex.

Exam	Connected exams	Degree
A	B, C, D	3
B	A, C, D	3
C	A, B, E	3
D	A, B, E	3
E	C, D	2

(b) First exam selection using MRV and degree heuristic

The Minimum Remaining Values (MRV) heuristic selects the exam with the smallest domain.

Exam	Initial domain	Domain size	Degree
A	12	2	3
B	123	3	3

C	123	3	3
D	123	3	3
E	12	2	2

A and E both have the smallest domain size of 2. The degree heuristic breaks the tie by choosing A, because A has the higher degree (3 compared with 2).

Using ascending value order, assign A = 1.

Forward checking: Since A is connected to B, C and D, remove slot 1 from their domains.

Exam	Domain after assigning A = 1
A	1
B	23
C	23
D	23
E	12

(c) Continue the search using MRV, degree heuristic and forward checking

After assigning A = 1, the remaining domains are:

- B = 3
- C = 3
- D = 3
- E = 2

Step 2: Select B

B, C, D and E all have two available values. Their degrees among the unassigned exams are tied at 2, so the alphabetical tie-break selects B.

Assign B = 2.

Forward checking removes slot 2 from C and D because both share students with B.

Exam Updated domain

A 1

B 2

C	3
D	3
E	12

Step 3: Select C

C and D have singleton domains, so both qualify under MRV. Their degrees among the remaining unassigned exams are tied, so choose C alphabetically.

Assign C = 3. No further domain values are removed.

Step 4: Select D

D is the next exam with a singleton domain.

Assign D = 3. E remains 2 because neither C nor D uses slot 1 or 2.

Step 5: Select E

Assign E = 1, the smallest available value.

The complete assignment is:

Exam	Assigned slot
A	1
B	2
C	3
D	3
E	1

Verification of all constraints

Constraint	Assigned slots	Valid?
A–B	1 and 2	Yes
A–C	1 and 3	Yes
A–D	1 and 3	Yes
B–C	2 and 3	Yes
B–D	2 and 3	Yes

C–E	3 and 1	Yes
D–E	3 and 1	Yes

The assignment is valid. A and E are also restricted to slots 1 or 2, as required.

(d) Interpretation

- MRV: Selects the exam with the fewest available slots, reducing the chance of reaching a dead end late in the search.
- Degree heuristic: Breaks ties by prioritizing the exam that constrains the largest number of other unassigned exams.
- Forward checking: Removes values that conflict with each assignment, exposing restrictions early and preventing unnecessary exploration.

If any domain becomes empty, forward checking signals a dead end: no legal value remains for that exam under the current assignments. The algorithm must backtrack and try a different value for an earlier exam.

Question 4: Board-Game AI – Minimax, Alpha-Beta Pruning and Evaluation Function

Topic: Game playing, adversarial search, evaluation functions | 15 minutes

A game-playing agent (MAX) searches two plies ahead: MAX chooses one of three moves (B, C, D), then MIN replies with one of three moves. Positions at depth 2 are scored by the evaluation function

Eval = 2 × M + 1 × Mo

where M is the material advantage of MAX and Mo is the mobility advantage of MAX. The leaf features (M, Mo) are given for the leaves in left-to-right order:

MIN node	Leaf 1 (M, Mo)	Leaf 2 (M, Mo)	Leaf 3 (M, Mo)
B	(1, 1)	(5, 2)	(3, 2)
C	(1, 0)	(1, 2)	(2, 2)
D	(6, 2)	(2, 1)	(0, 2)

(a) Compute the Eval value of all nine leaves.

(b) Run plain minimax. Give the backed-up value of B, C and D, the value at the root and the move MAX should play.

(c) Run alpha-beta pruning on the same tree (left to right). Show α and β at each node, mark every pruned branch, and state the root value and the number of leaves evaluated.

(d) If the leaves under C were ordered (6, 4, 2) instead, how many leaves would alpha-beta evaluate? Explain the effect of move ordering and why pruning never changes the minimax value.

(a) Compute the evaluation values of all nine leaves

The evaluation function is:

$$\text{Eval}=2M+Mo$$

where M is the material advantage and Mo is the mobility advantage of MAX.

MIN node	Leaf	(M,Mo)	Calculation	Eval
B	1	(1, 1)	$2(1)+1$	3
B	2	(5, 2)	$2(5)+2$	12
B	3	(3, 2)	$2(3)+2$	8
C	1	(1, 0)	$2(1)+0$	2
C	2	(1, 2)	$2(1)+2$	4
C	3	(2, 2)	$2(2)+2$	6
D	1	(6, 2)	$2(6)+2$	14
D	2	(2, 1)	$2(2)+1$	5
D	3	(0, 2)	$2(0)+2$	2

(b) Run plain minimax

MAX chooses the move with the highest value, while MIN chooses the lowest-valued reply.

At node B (MIN):

$$B=\min(3,12,8)=\boxed{3}$$

At node C (MIN):

$$C=\min(2,4,6)=\boxed{2}$$

At node D (MIN):

$$D=\min(14,5,2)=\boxed{2}$$

At the root (MAX):

$$\text{Root}=\max(3,2,2)=\boxed{3}$$

Move	Backed-up value
B	3
C	2
D	2

Best move for MAX: B, with a minimax value of 3.

(c) Run alpha-beta pruning from left to right

Alpha-beta pruning maintains:

- α: The best value MAX can guarantee so far.
- β: The best value MIN can guarantee so far.

Initially, $\alpha=-\infty$ and $\beta=+\infty$.

Step-by-step traversal

1. Explore B (MIN)

The leaf values are 3, 12 and 8.

- First leaf: $\beta=\min(+\infty,3)=3$.
- Second leaf: $\beta=\min(3,12)=3$.
- Third leaf: $\beta=\min(3,8)=3$.

B returns 3. At the root, MAX updates $\alpha=\max(-\infty,3)=3$.

2. Explore C (MIN)

C inherits $\alpha=3,\ \beta=+\infty$.

- First leaf: 2, so $\beta=\min(+\infty,2)=2$.
- Now $\beta\leq\alpha$, because $2\leq3$.

The remaining two leaves under C, with values 4 and 6, are pruned.

C returns 2.

3. Explore D (MIN)

D inherits $\alpha=3,\ \beta=+\infty$.

- First leaf: 14, so $\beta=14$.
- Second leaf: 5, so $\beta=5$.

- Third leaf: 2, so $\beta=2$.

The cutoff condition is reached at the last leaf, so there are no further leaves under D to prune. D returns 2.

The root remains:

$$\max(3,2,2)=\boxed{3}$$

Alpha-beta summary

Node	Initial α	Initial β	Leaf values evaluated	Return value
B	$-\infty$	$+\infty$	3, 12, 8	3
C	3	$+\infty$	2; then prune 4, 6	2
D	3	$+\infty$	14, 5, 2	2
Root (MAX)	$-\infty$	$+\infty$	Child values 3, 2, 2	3

Final alpha-beta results

Root minimax value

3

Leaves evaluated

7 of 9

Pruned leaves: C's second and third leaves (values 4 and 6).

(d) Effect of changing the leaf order under C

Suppose the leaves under C are ordered as (6, 4, 2) instead of (2, 4, 6).

C inherits $\alpha=3$.

1. Evaluate 6: $\beta=6$. No cutoff.
2. Evaluate 4: $\beta=4$. No cutoff because $4>3$.
3. Evaluate 2: $\beta=2$. Now $\beta\leq\alpha$, so the cutoff occurs after the third leaf.

Therefore, all three leaves under C must be evaluated. No leaves are pruned under C in this ordering.

- B evaluates 3 leaves.
- C evaluates 3 leaves.
- D evaluates 3 leaves.

Total leaves evaluated:

$$ 3+3+3=\boxed{9} $$

The root value remains 3, and MAX still chooses B.

Explanation: Move ordering affects the effectiveness of alpha-beta pruning. Finding a low value early at a MIN node can trigger a cutoff sooner, while finding it last may require evaluating every leaf. However, alpha-beta pruning never changes the minimax value because it eliminates branches that cannot affect the final decision.